



Dhaka International University

Lab Report details:

Lab Report Topic : **Write the program to solve FCFS scheduling algorithm using C or C++**

Lab Report No : 01

Course Title : Operating System

Course Code : 0613-301

Submitted By:	Submitted To:
S M Nabil Ausaf Roll: 01 Reg No : CS-D-91-23-124492	Md. Nazmus Sakib, Lecturer of CSE Department

Date of Submission:31.10.25

Lab No: 01**Lab Name: First Come First Serve (FCFS) Scheduling Algorithm****Objectives:**

- To comprehend and apply operating systems' process scheduling concept.
- To schedule CPUs using the First Come First Serve (FCFS) algorithm.
- To determine the average turnaround time and average waiting time for a specific set of procedures.

Background Study:

The process of choosing which process will use the CPU next is known as CPU scheduling.

The process that arrives first is carried out first in the FCFS (First Come First Serve) scheduling algorithm. It is non-preemptive, which means that once a process begins, it will continue until it is finished.

Characteristics:

- Easy to use and straightforward.
- The convoy effect, where a lengthy process delays all others, may occur.
- Procedures are carried out in the order that they arrive.

Example:

Given processes with burst times:

Process	Burst Time (ms)
P1	24
P2	3
P3	3

All arrive at time **0**.

Algorithm:

1. Start
2. Input number of processes and their burst times.
3. Compute Waiting Time (WT):
 - $WT[i] = \text{sum of burst times of all previous processes}$
4. Compute Turnaround Time (TAT):
 - $TAT[i] = BT[i] + WT[i]$
5. Calculate average waiting time and average turnaround time.
6. Display results.
7. Stop

CODE:

```
Start here X  Untitled1.c X
1  #include <stdio.h>
2  int main() {
3      int n = 3;
4      int burst_time[] = {24, 3, 3};
5      int waiting_time[3], turnaround_time[3];
6      float avg_wt = 0, avg_tat = 0;
7      waiting_time[0] = 0; // first process has no waiting time
8      // Calculate waiting times
9      for (int i = 1; i < n; i++) {
10         waiting_time[i] = burst_time[i - 1] + waiting_time[i - 1];
11     }
12     // Calculate turnaround times
13     for (int i = 0; i < n; i++) {
14         turnaround_time[i] = burst_time[i] + waiting_time[i];
15         avg_wt += waiting_time[i];
16         avg_tat += turnaround_time[i];
17     }
18     avg_wt /= n;
19     avg_tat /= n;
20     printf("Process\tBurst Time\tWaiting Time\tTurnaround Time\n");
21     for (int i = 0; i < n; i++) {
22         printf("P%d\t\t%d\t\t%d\t\t%d\n", i + 1, burst_time[i], waiting_time[i], turnaround_time[i]);
23     }
24     printf("\nAverage Waiting Time: %.2f", avg_wt);
25     printf("\nAverage Turnaround Time: %.2f\n", avg_tat);
26     return 0;
27 }
28
```

OUTPUT:

Process	Burst Time	Waiting Time	Turnaround Time
P1	24	0	24
P2	3	24	27
P3	3	27	30

Average Waiting Time: 17.00

Average Turnaround Time: 27.00

Discussion:

The FCFS scheduling algorithm was successfully implemented in this lab.

All processes arrive at time 0, so they are carried out in the following order: P1 → P2 → P3.

Despite being simple to implement, the algorithm is not the best at reducing waiting times, particularly when lengthy processes come up first.

The fundamental idea behind CPU scheduling in an operating system is illustrated by this experiment.